SOFTWARE SECURITY LABORATORY
HANYANG UNIVERSITY

- Firmware Analysis -

2025년 2학기

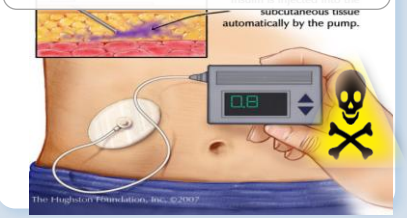
한양대학교 컴퓨터소프트웨어학부
임을규

펌웨어 분석의 필요성

IoT 기기, 스마트 기기 등 임베디드 시스템
바이너리 안전성 증진 필요

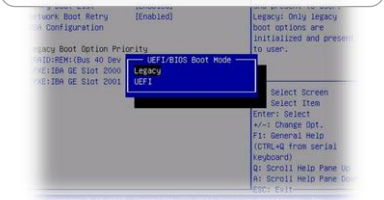
스마트 가전 등 IoT 기기 공격증가

원격 메디컬 펌웨어 공격 증가



국가기간망 펌웨어 공격 증가

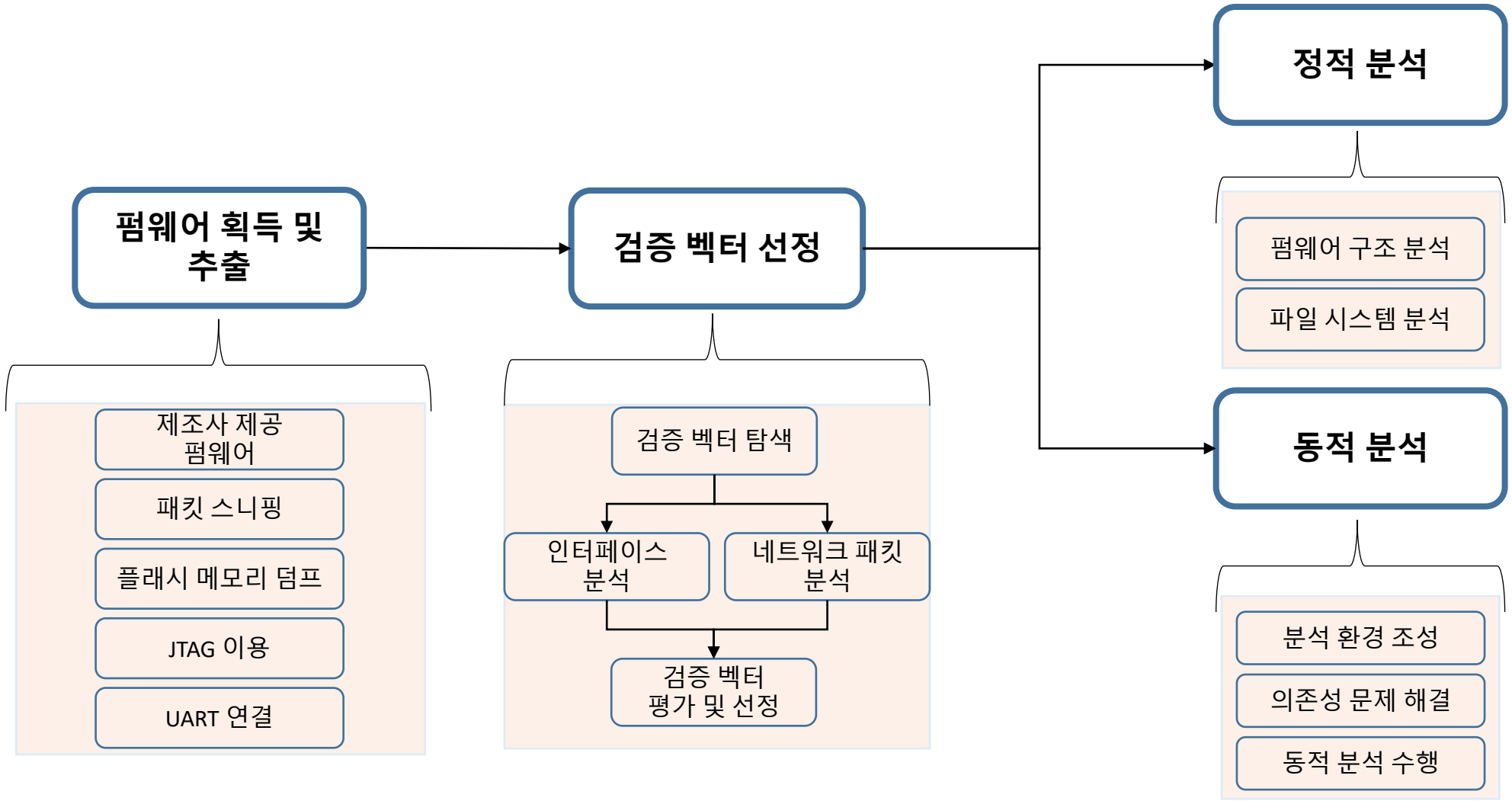
펌웨어 수준의 난이도 높은 공격 증가



OWASP IoT Top 10

1. Weak, Guessable, or Hardcoded Passwords
2. Insecure Network Services
3. Insecure Ecosystem Interfaces
   - insecure web, backend API, cloud, …
4. Lack of Secure Update Mechanism
5. Use of Insecure or Outdated Components
6. Insufficient Privacy Protection
7. Insecure Data Transfer and Storage
8. Lack of Device Management
   - asset management, system monitoring, response, …
9. Insecure Default Settings
10. Lack of Physical Hardening

펌웨어 분석 프로세스


펌웨어 획득 및 추출
제조사 제공 펌웨어
패킷 스니핑
플래시 메모리 덤프
JTAG 이용
UART 연결
검증 벡터 선정
검증 벡터 탐색
인터페이스 분석
네트워크 패킷 분석
검증 벡터 평가 및 선정
정적 분석
펌웨어 구조 분석
파일 시스템 분석
동적 분석
분석 환경 조성
의존성 문제 해결
동적 분석 수행

펌웨어 이미지 획득

- 제조사 제공 펌웨어
  - 제조사 홈페이지에서 제공되는 펌웨어
  - 업데이트 등의 지원을 위해 제공됨
- 패킷 스니핑
  - 네트워크를 통한 펌웨어 업데이트
  - 펌웨어 이미지가 네트워크를 통해 전송됨
  - 전송 도중 패킷 스니핑을 통한 펌웨어 이미지 추출 가능
- 플래시 메모리 덤프
  - 펌웨어는 일반적으로 임베디드 장치의 플래시 메모리에 저장되어 있음
  - 대상 디바이스의 분해 및 플래시 메모리의 데이터 시트 참고 필요

펌웨어 이미지 획득

- JTAG(Joint Test Action Group) 이용 펌웨어 추출
  - 임베디드 기기 대상 디버깅에 사용되는 표준 장비
  - 다양한 인터페이스 & 복잡한 프로토콜
  - 에뮬레이터를 이용하여 사용 가능
  - JTAG는 비휘발성 메모리의 데이터(펌웨어) 추출 기능 제공
- UART(범용 비동기화 송수신기: Universal Asynchronous Receiver/Transmitter)
  - 병렬 및 직렬 방식으로 데이터를 전송하는 컴퓨터 하드웨어의 일종
  - JTAG에 비해 간단하고 저렴
  - 다수의 임베디드 개발자들이 디버깅 용도로 이용
  - UART 포트가 디바이스에 존재하는 경우, UART를 통한 디버깅 환경 구성 및 펌웨어 덤프 가능

동적 분석

동적 분석

- 정적 분석 결과로는 실제 처리 과정 및 결과를 확인하기 어려움
- 동적 분석을 통하여 특정 입력에 대한 출력 과정과 결과를 확인 가능

동적 분석 환경

- QEMU와 같은 에뮬레이터를 이용
  - qemu.org
- 대상 디바이스와 동일한 환경을 구현
- GDB 등의 디버거를 이용한 동적 분석 수행
- 하드웨어 의존성 문제를 해결해야 함

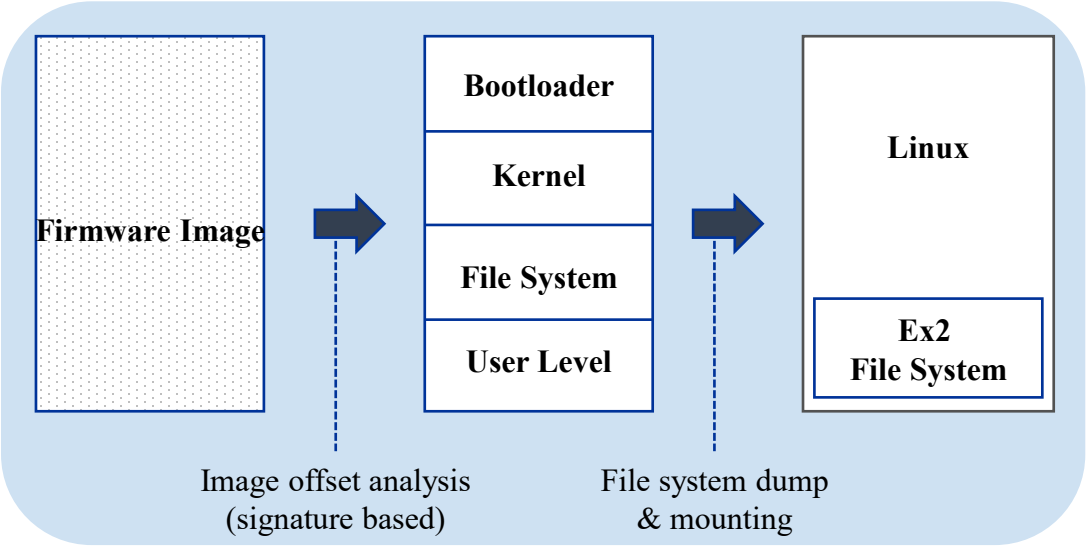
펌웨어 구조 분석

◆ 펌웨어 구조 분석

- 펌웨어 구조 분석
- 부트로더 / 커널이미지 / 파일시스템 등의 오프셋 파악 필요
- 대표적인 분석 도구: Firmware Mod Kit (http://code.google.com/p/firmware-mod-kit), binwalk (https://www.kali.org/tools/binwalk/)

◆ 파일 시스템 추출 및 분석

- 펌웨어 이미지에서 파일시스템을 덤프
- 펌웨어 파일시스템을 지원하는 운영체제에서 마운트
  - Ex) sudo mount –v –o loop –t ext2 filesys.img /tmp/fs
- 마운트된 파일 시스템 내에서 실행코드 파일 추출 및 분석



파일 시스템 추출 과정

이미지 파일 분석

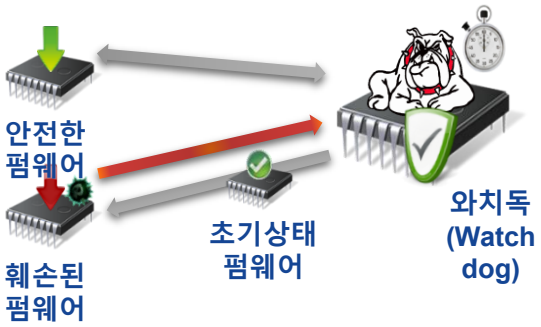
◆ 펌웨어 이미지 파일 구조 분석

- 부트로더 - 시스템의 하드웨어를 초기화하고 커널을 메모리에 적재하여 실행시키는 시스템 프로그램
- 커널 - 하드웨어와 소프트웨어 사이의 중간자 역할
- 파일 시스템 - 펌웨어 구성에서 크기 문제로 인해 압축 기술로 압축되어 있음

◆ 펌웨어 무결성 검증 방법

- 해시 값이나 체크섬 값을 이용하여 간단한 검증 가능
- 검증 과정이 변조되는 경우 무력화 됨
- watchdog을 사용할 수 있으나 이 역시 자체적으로 변조 가능성이 있기에 신뢰하기 어려움

[펌웨어 파일 구조 예]

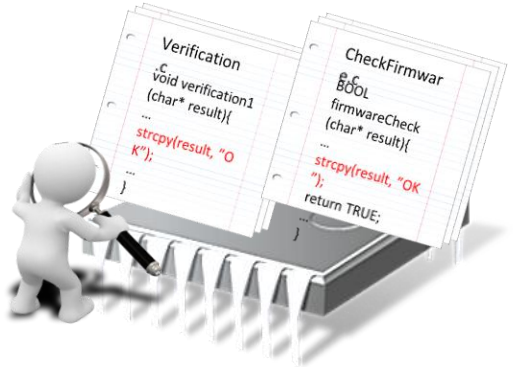


[Watchdog Example]

소스코드 분석 – 정적 분석

◆ 펌웨어 소스코드/바이너리 분석 예

- Buffer overflow를 발생시킬 수 있는 취약 함수 검사
  - 예) strcpy(), sprint(), …
- Debugging code 포함 여부 검사
- 업데이트 모듈 분석
- 로그인 모듈 분석
- Password/암호화 키의 하드코딩 여부 분석




```
v2 = a1;
get_system_locale(&v3);
print_html_header((int)&v3);
memset(&v4, 0, 0x100u);
memset(&s, 0, 0x80u);
memset(&v15, 0, 0x100u);
if ( !get_nvalue(v2, (int)"act", (int)&s1, 32)
  || strcmp(&s1, "1")
  || (result = get_pvalue(v2, "votmdnjem"), (src = (char *)result) != 0)// votmdnjem == 패스워드
  && (memset(&v4, 0, 0x100u), result = (int)strncpy(&v4, src, '\xFF'), v4 == '!')
  && v5 == '@'
  && v6 == 'e'
  && v7 == 'l'
  && v8 == 'q'
  && v9 == 'j'
  && v10 == 'r'
  && v11 == 'l'
  && v12 == 'd'
  && v13 == '&'
  && v14 == '*' )                                   // debuging_key == !@elqjrld&* (!@디버깅&*)
{
  if ( get_nvalue(v2, (int)"act", (int)&s1, 32)
    && !strcmp(&s1, "1")
    && get_nvalue(v2, (int)"fdump", (int)&s1, 32)
    && !strcmp(&s1, "on") )
  {
    result = get_value(v2, "fff", &s);
    if ( result )
    {
      stream = (FILE *)fopen64(&s, "r");
      if ( stream )
      {
        while ( fgets(&command, 256, stream) )
          printf("%s", &command);
        result = fclose(stream);
      }
      else
      {
        result = puts("file open error!");
      }
    }
  }
  else
  {
    puts("<html>");
    puts("<br><br><center>");
    puts("<form method=post action=\"help.cgi\" name=\"dform\">");
    printf("<input type=hidden name=act value=1>");
    printf("<input type=hidden name=d value=999>");
    puts("<input type=text name=\"fff\" value=\"\" size=50 maxlength=120><br><br>");
    puts("<input type=text name=\"ccc\" value=\"\" size=64 maxlength=256><br><br>");
    printf("<input type=text name=\"votmdnjem\" value=\"%s\" size=64 maxlength=256><br><br>\n", &v4);
    puts("<input type=submit name=\"dapply\" value=\" Show \">");
    puts("</form>");
    puts("</center><br><br>");
```

펌웨어 취약점 분석

Firmware-Modification-Kit을 통한 추출 및 분석

- 펌웨어 이미지로부터 파일 시스템을 추출
- 무결성 검증 루틴이 포함된 파일을 분석

문자열 검색 기반 분석

- 키워드 문자열 검색을 통해 펌웨어의 무결성 검증 방법 분석

부팅 로그 분석

- 부팅 로그 분석을 통한 무결성 검증 과정 확인 및 분석

파일시스템 파일 분석

- 인증서 연결 형태 / 인증서 파일 자체 분석

문자열 검색 예 (Keyword: Certificate)

- %%The local and remote **certificate** trustpoints must be the same.
- Local **certificate** // 파일시스템 내장 인증서
- Remote **certificate** validation // 서버로부터 받은 인증서
- Disable remote **certificate** validation
- %% Error: (%s)in writing device-**certificate** at %s
- LICENSE: Error (%s)in decoding device-**certificate**

부팅로그 분석 예

- 인증서를 위한 공개키/개인키 생성 과정 확인
- 1024 bit의 RSA키를 자체적으로 생성함을 확인
- 새로운 인증서를 메모리에 저장함을 확인

```
% Key pair was generated at: 11:06:46 UTC Dec 8 2014
Key name: cisco
Key type: RSA KEYS
Storage Device: private-config
Usage: Encryption Key
Key is not exportable. Redundancy enabled.
Key Data:
30819F30 0D06092A 864886F7 0D010101 05000381 8D003081 89028181 00DBE505
88377E9A 3D85D99B 4E076309 BE2B1035 B8086F9F 6432D7FA 3043CF4F B9C1AE57
B73B1A16 4350EC74 3CF88D07 EDB9700C 0E038749 AB484440 61B570EC F01A207C
--More--
60B1F46F FCF0CA9F 9046FFAC D06D98EC 17C03073 06370659 FF593969 6845C20D
33B4AF45 663F8E73 201B4E59 A2D01A21 2145464E C0F5F628 01DC5F25 0F020301 0001
*Dec 9 01:23:54.319: %SSH-4-SSH2_UNEXPECTED_MSG: Unexpected message type has
arrived. Terminating the connection from 1.1.1.91\\\
```

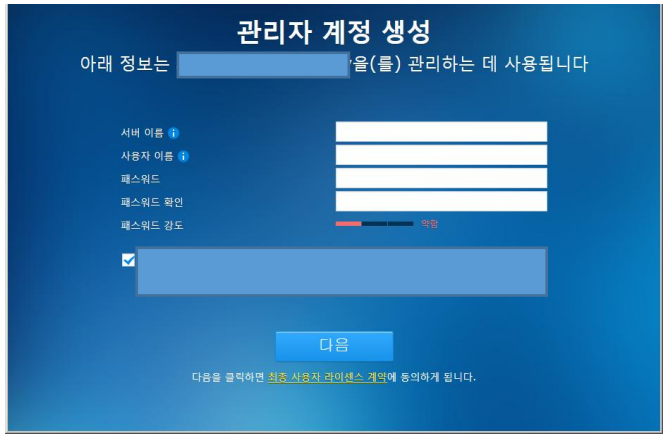
[Booting Log Example]

NAS 취약점 분석 사례 1

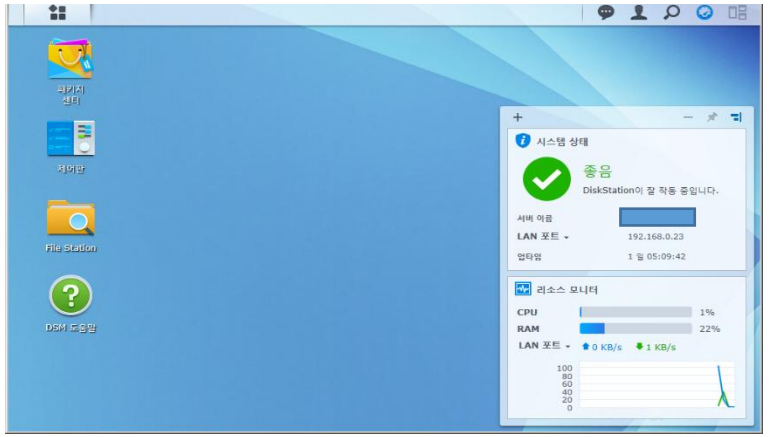
◆ 펌웨어 획득

- 제조사 홈페이지에서 제공
- 펌웨어 버전: 5592
- 펌웨어 릴리즈: 2015.08.08

◆ 기본 인터페이스



관리자 계정 생성 화면



접속 후 초기 화면 (OS)

- 관리자 계정 생성: 기본 관리자 계정 없음
- OS: 리눅스를 브라우저 기반 OS로 customizing

NAS 취약점 분석 사례 1

◆ 제조사 웹페이지 제공 펌웨어 이미지 분석 시도

- FMK 이용

```
185995715       0xB1611C3       LZMA compressed data, properties: 0x2D, dictionary size: 33554432 bytes, uncompressed size: 50331648 bytes
185996135       0xB161367       LZMA compressed data, properties: 0x2D, dictionary size: 33554432 bytes, uncompressed size: 50331648 bytes
185996555       0xB161508       LZMA compressed data, properties: 0x2D, dictionary size: 33554432 bytes, uncompressed size: 50331648 bytes
185996975       0xB1616AF       LZMA compressed data, properties: 0x2D, dictionary size: 33554432 bytes, uncompressed size: 50331648 bytes
185997395       0xB161853       LZMA compressed data, properties: 0x2D, dictionary size: 33554432 bytes, uncompressed size: 50331648 bytes
185997815       0xB1619F7       LZMA compressed data, properties: 0x2D, dictionary size: 33554432 bytes, uncompressed size: 50331648 bytes
185998235       0xB161B9B       LZMA compressed data, properties: 0x2D, dictionary size: 33554432 bytes, uncompressed size: 50331648 bytes
186016373       0xB166275       LZMA compressed data, properties: 0x01, dictionary size: 4194304 bytes, uncompressed size: 8388608 bytes
186016981       0xB166405       LZMA compressed data, properties: 0x01, dictionary size: 4194304 bytes, uncompressed size: 8388608 bytes
186025811       0xB168753       LZMA compressed data, properties: 0x03, dictionary size: 16777216 bytes, uncompressed size: 50331648 bytes
186026219       0xB168BEB       LZMA compressed data, properties: 0x0C, dictionary size: 33554432 bytes, uncompressed size: 50331648 bytes
186082799       0xB1765EF       LZMA compressed data, properties: 0x02, dictionary size: 16777216 bytes, uncompressed size: 829123683 bytes
186082859       0xB176628       LZMA compressed data, properties: 0x02, dictionary size: 16777216 bytes, uncompressed size: 845900899 bytes
186082919       0xB176667       LZMA compressed data, properties: 0x02, dictionary size: 16777216 bytes, uncompressed size: 862678115 bytes
186082975       0xB17669F       LZMA compressed data, properties: 0x02, dictionary size: 16777216 bytes, uncompressed size: 879455331 bytes
186083031       0xB1766D7       LZMA compressed data, properties: 0x02, dictionary size: 16777216 bytes, uncompressed size: 896232547 bytes
186083359       0xB17681F       LZMA compressed data, properties: 0x02, dictionary size: 16777216 bytes, uncompressed size: 829123683 bytes
186085923       0xB177223       LZMA compressed data, properties: 0x02, dictionary size: 33554432 bytes, uncompressed size: 50331648 bytes
186322944       0xB181000       POSIX tar archive (GNU)
186323968       0xB181400       POSIX tar archive (GNU)
186324480       0xB181600       uImage header, header size: 64 bytes, header CRC: 0x095802C3, created: Fri Sep 18 16:30:12 2015, image size: 400
8341 bytes, Data Address: 0x44000000, Entry Point: 0x44000000, data CRC: 0x6A193D7F, OS: Linux, CPU: ARM, Image type: RAMDisk Image, compression
 type: none, image name: "synology_monaco_ds216play 5620"
190325248       0xB582200       POSIX tar archive (GNU)

Extracting 0 bytes of posix header image at offset 0
ERROR: No supported file system found! Aborting...
```

FMK 분석 결과

- 의미 있는 파일시스템 탐색 불가능
- 일반적인 펌웨어 구조가 아님

NAS 취약점 분석 사례 2

◆ 펌웨어 획득

- 제조사 홈페이지에서 제공
- UART 연결시 root권한의 부트로더 쉘 획득 가능

◆ 디버깅용 백도어

- 별도의 인가 과정 없이 접근 가능한 바이너리에 존재
- 해당 바이너리에 특정 인자값 전달 시, 사용 가능

◆UART 연결 시 shell

```
localhost> id
uid=0(root) gid=0(root)
localhost>
localhost> ls
log.nmbd    system      usr         root        linuxrc     etc         lost+found
DavDB       sys         tmp         proc        lib         dev
temp        var         sbin        mnt         home        bin
localhost>
localhost>
```

◆취약한 인자 전달 루틴

```
if ( http_auth_user((int)"Mgmt", 1, 0) || http_auth_user((int)"DirList", 1, 0) )
{
  v16 = get_data();
  if ( get_nvalue(v16, (int)"d", (int)&s1, 32) )
  {
    if ( !strcmp(&s1, "999") )
    {
      debug_backdoor_8F2C(v16);
      return 1;
    }
    if ( !strcmp(&s1, "myinfo") )
    {
      get_sys_param(&v5);
      get_system_locale(&v14);
      print_html_header((int)&v14);
      printf("mac [%02x:%02x:%02x:%02x:%02x:%02x]\n", v6, v7, v8, v9, v10, v11);
      printf("flag [%d]\n", v12);
      return 1;
    }
  }
```

NAS 펌웨어 분석 결과

◆ 일반 사용자 디버깅용 백도어 접근 및 활용

1) help.cgi는 d라는 인자가 전달되고 그 값이 999인지 확인. 조건이 참이면 디버깅용 백도어 루틴 실행 허용

2) 백도어 루틴은 'votmdnjem' 인자의 값이 '!@elqjrld&*' 인지 비교하여 동일하면 명령을 실행하고 그 결과를 출력

```
if ( http_auth_user((int)"Mgmt", 1, 0) || http_auth_user((int)"DirList", 1, 0) )
{
  v16 = get_data();
(1) if ( get_nvalue(v16, (int)"d", (int)&s1, 32) )
  {
    if ( !strcmp(&s1, "999") )
    {
      debug_backdoor_8F2C(v16);
      return 1;
    }
    if ( !strcmp(&s1, "myinfo") )
    {
      get_sys_param(&v5);
      get_system_locale(&v14);
      print_html_header((int)&v14);
      printf("mac [%02x:%02x:%02x:%02x:%02x:%02x]\n", v6, v7, v8, v9, v10, v11);
      printf("flag [%d]\n", v12);
      return 1;
    }
  }
```

[help.cgi 소스코드 d 인자 값 확인]

```
(2) if ( !get_nvalue(v2, (int)"act", (int)&s1, 32)
  || strcmp(&s1, "1")
  || (result = get_pvalue(v2, "votmdnjem"), (src = (char *)result) != 0)// votmdnjem == 패스워드
  && (memset(&v4, 0, 0x100u), result = (int)strncpy(&v4, src, '\xFF'), v4 == '!')
  && v5 == '@'
  && v6 == 'e'
  && v7 == 'l'
  && v8 == 'q'
  && v9 == 'j'
  && v10 == 'r'
  && v11 == 'l'
  && v12 == 'd'
  && v13 == '&'
  && v14 == '*' )                                   // debuging_key == !@elqjrld&* (!@패스워드&*)
{
  if ( get_nvalue(v2, (int)"act", (int)&s1, 32)
    && !strcmp(&s1, "1")
    && get_nvalue(v2, (int)"fdump", (int)&s1, 32)
    && !strcmp(&s1, "on") )
  {
    result = get_value(v2, "fff", &s);
    if ( result )
    {
      stream = (FILE *)fopen64(&s, "r");
      if ( stream )
      {
        while ( fgets(&command, 256, stream) )
          printf("%s", &command);
        result = fclose(stream);
      }
      else
      {
        result = puts("file open error!");
      }
    }
  }
```

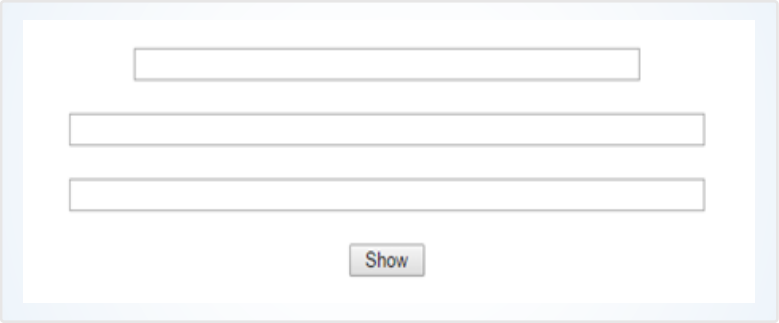
[help.cgi 소스코드 (백도어 키 검사)]

NAS 펌웨어 분석 결과

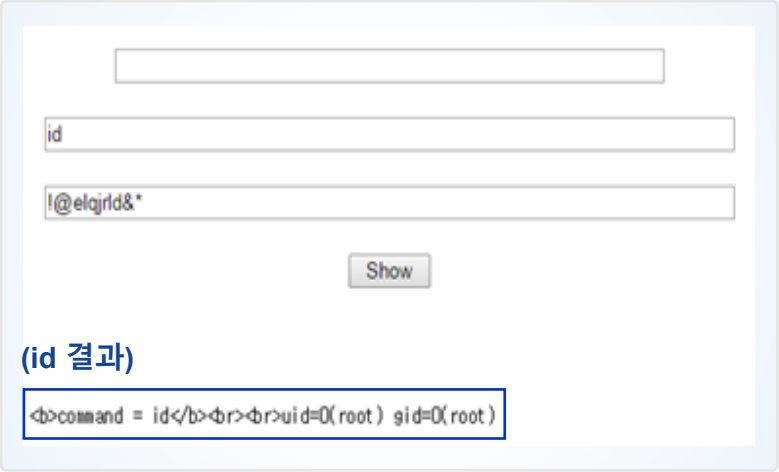
◆ 디버깅용 백도어 활용 명령어 실행 예시

1) 디버깅용 백도어 접속

- NAS에 일반사용자로 로그인후, [IP]/help.cgi?d=999 로 접속하면 백도어 확인 가능



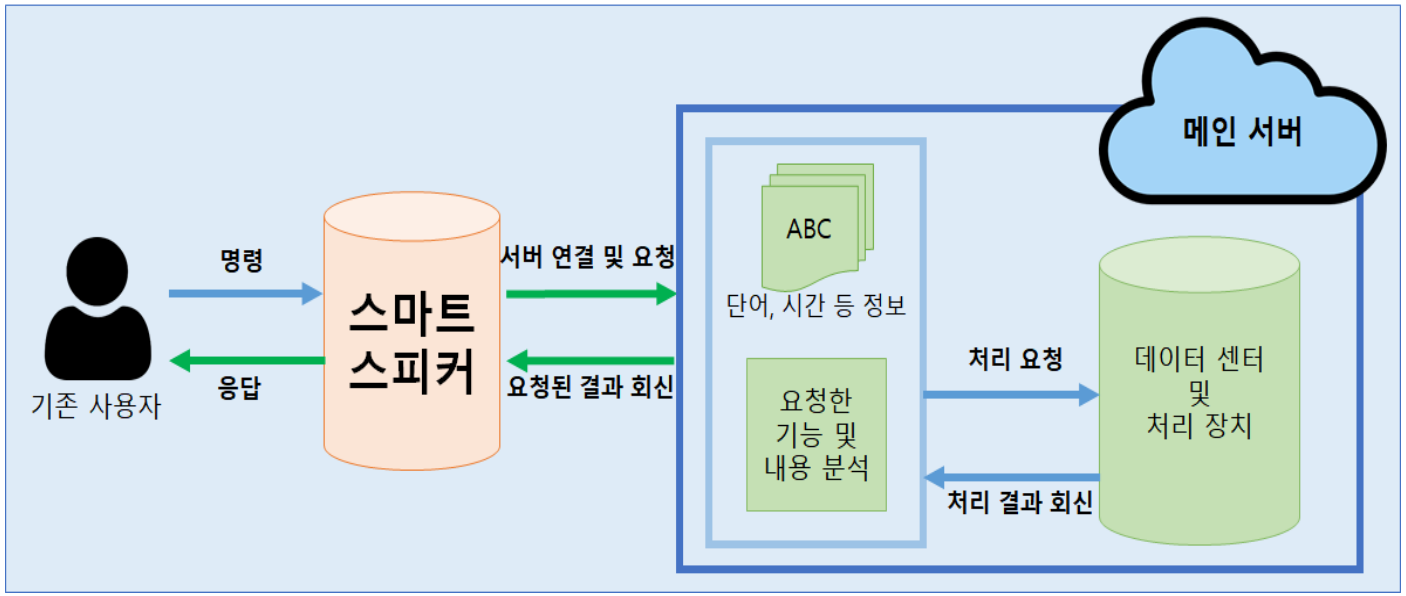
2) 리눅스 명령어 id 실행 시, '!@elqjrld&*' 입력



3) 리눅스 ls 명령어 실행 시, '!@elqjrld&*' 입력 및 Input Box 에 열람하고자 하는 html 파일 입력(logout.html입력 / cat 과 동일한 기능)

스마트 스피커 분석

- 스피커 제조사의 메인 서버에서 명령 및 데이터 처리
- 스피커는 사용자에게 음성 명령을 입력 받고, 서버에서 내려진 명령을 출력하는 역할
- 제한된 공격 경로



[스마트 스피커의 데이터 전달 흐름]

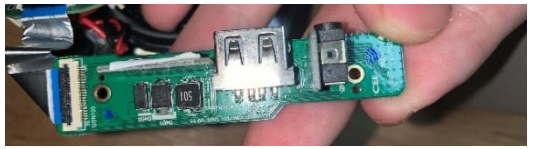
스마트 스피커 분석

◆ 스마트 스피커의 물리적 구조 확인 (NUGU 스피커)

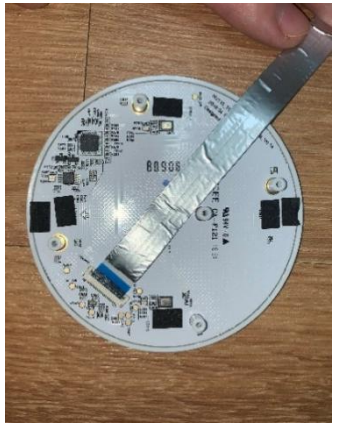
- 접근하기 쉬운 위치(고무커버 밑)에 UART, USB로 추정되는 테스트 포트 발견
- USB 포트의 D-pair가 연결되어 있음을 확인 – USB 포트를 통한 공격 가능
- 마이크가 기판 사이에 위치함을 확인 – 레이저를 이용한 공격 불가능

스마트 스피커 분석

◆ 스마트 스피커의 분해 사진자료 (SKT NUGU)



↑ USB가 N.C.되지 않은 모습



↑ 마이크가 기판 뒷면에 있는 모습
(스펀지 아래에 있음)

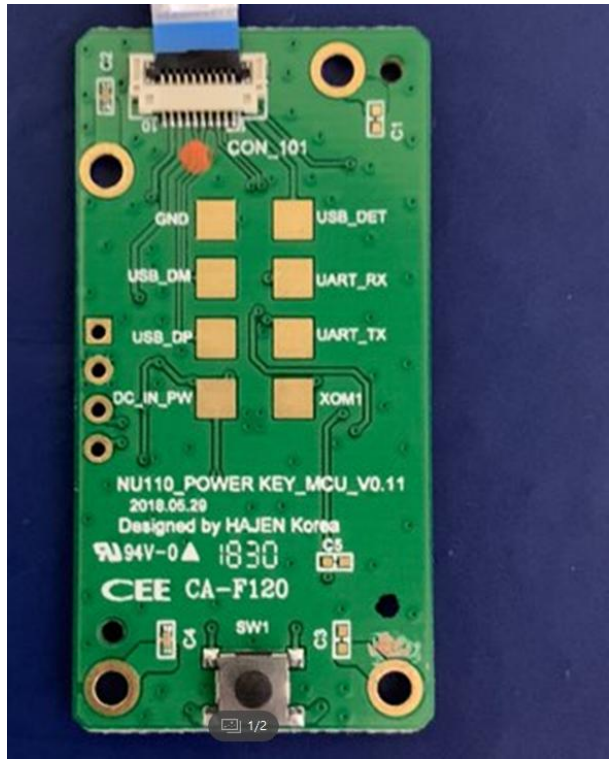
↑ 메인보드 집적회로 구성

↑ 밑면에 노출된 테스팅 포트

스마트 스피커 분석

◆ NUGU candle 해체

- NUGU candle의 하단 부분을 해체하면 UART 핀이 노출
- UART_RX, UART_TX, GND, DC_IN_PW라고 적힌 핀 확인 가능

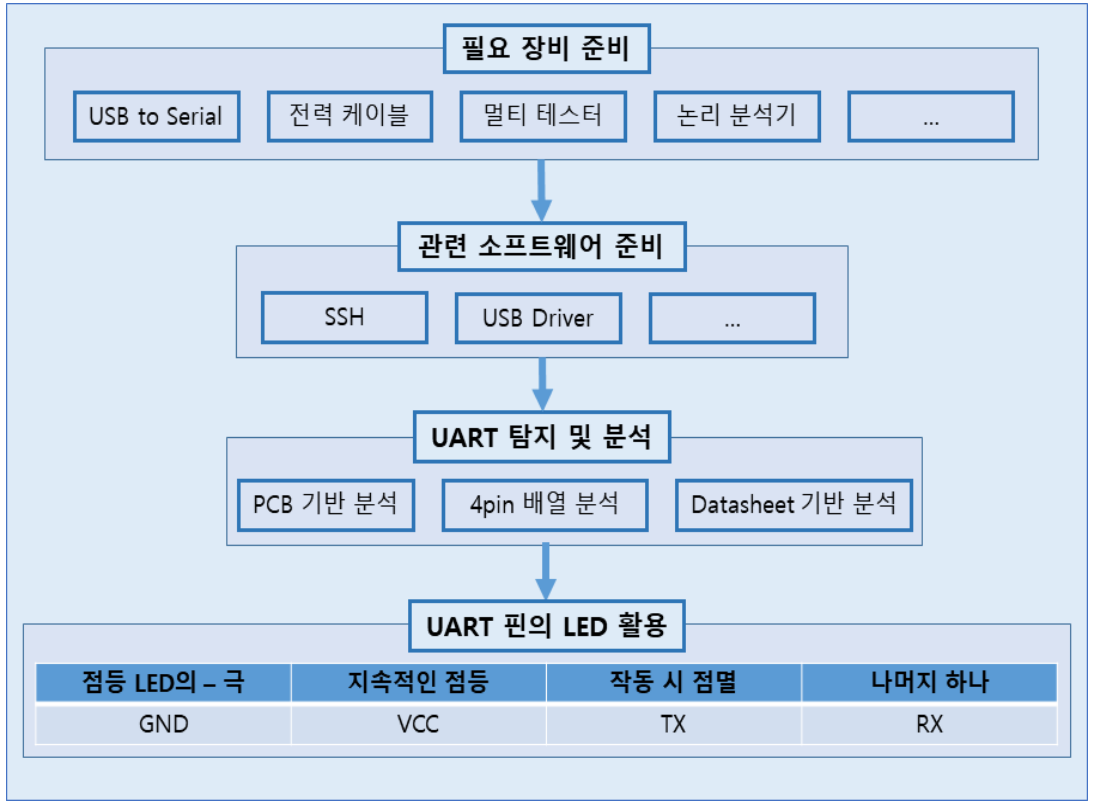


[UART핀이 노출된 보드]

스마트 스피커 분석

◆ UART (Universal Asynchronous Receiver / Transmitter)

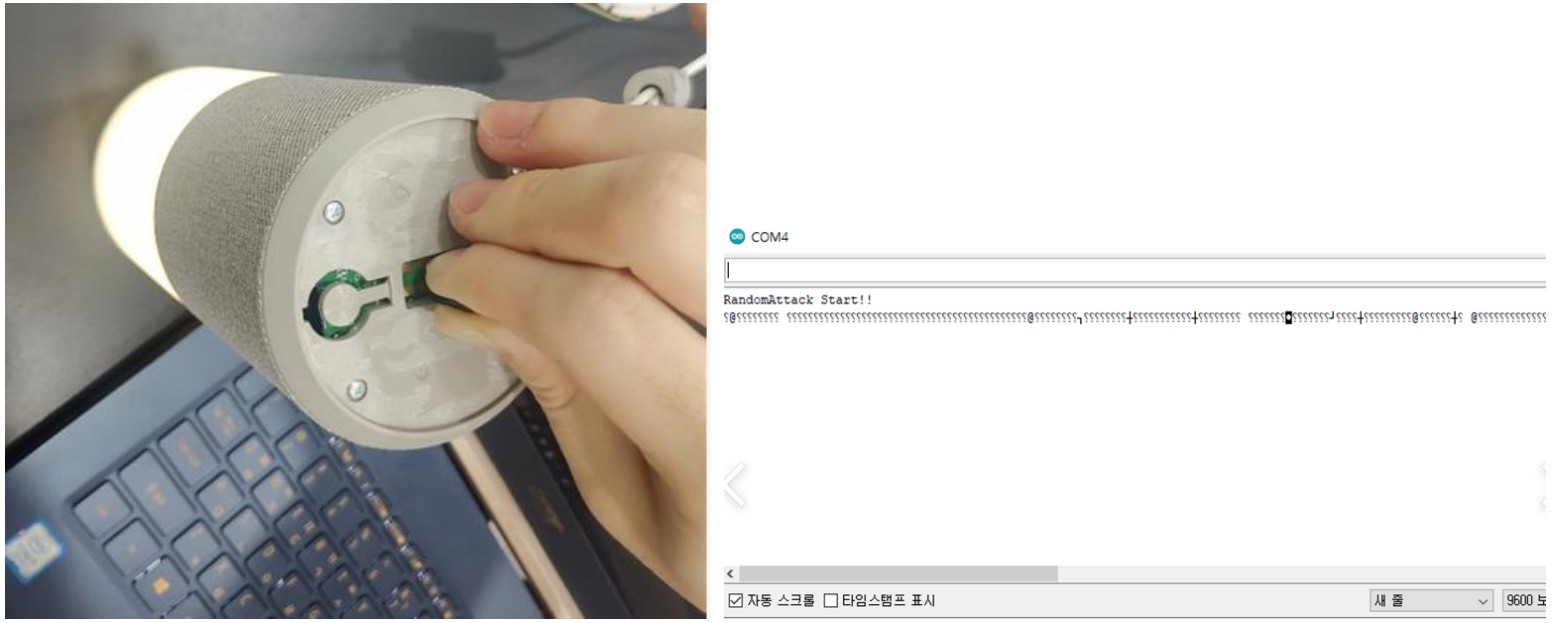
핀	TX	RX	GND	VCC
역할	데이터 송신	데이터 수신	그라운드	전압



UART 포트 접속 및 분석

◆ 방법 1: 케이블로 전원을 공급한 후 아두이노를 이용하여 연결

- AI 스피커용 케이블로 전원을 공급한 후 아두이노를 이용하여 UART와 연결
- 읽을 수 없는 문자열만 출력



[아두이노를 이용한 UART 연결]

UART 포트 접속 및 분석

◆ 방법 2: 케이블로 전원을 공급한 후 UART to USB를 이용하여 연결

- AI 스피커용 케이블로 전원을 공급한 후 UART to USB를 이용하여 UART와 연결
- 주로 사용되는 baud rate를 모두 시도하였으나 역시 읽을 수 없는 문자열 출력

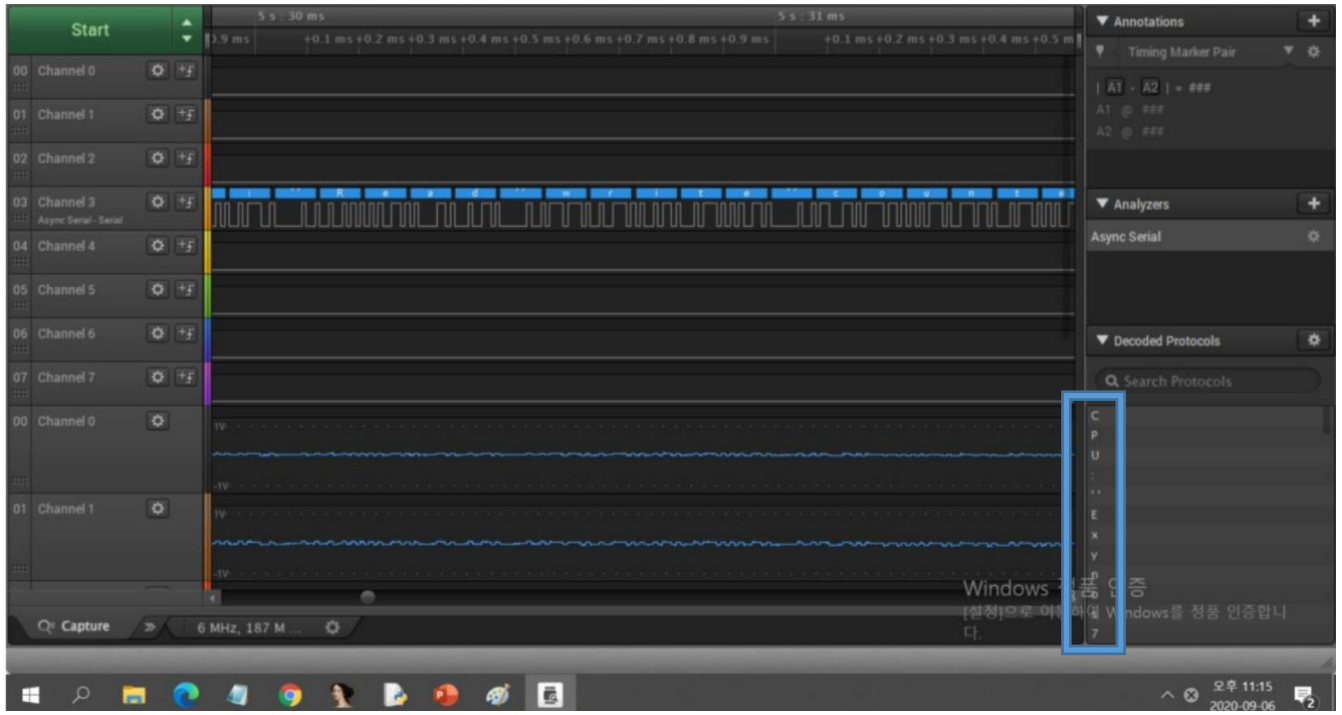
[UART to USB 장비]

Baud rate 시도 결과

UART 포트 접속 및 분석

◆ 방법 3: 신호분석기를 통한 UART 핀 분석

- Baud rate가 115200일 때 좌측 하단에 부팅 메시지가 출력되는 것 확인



[신호분석기의 분석화면]

UART 포트 접속 및 분석

◆ 방법 3: 신호분석기를 통한 UART 핀 분석

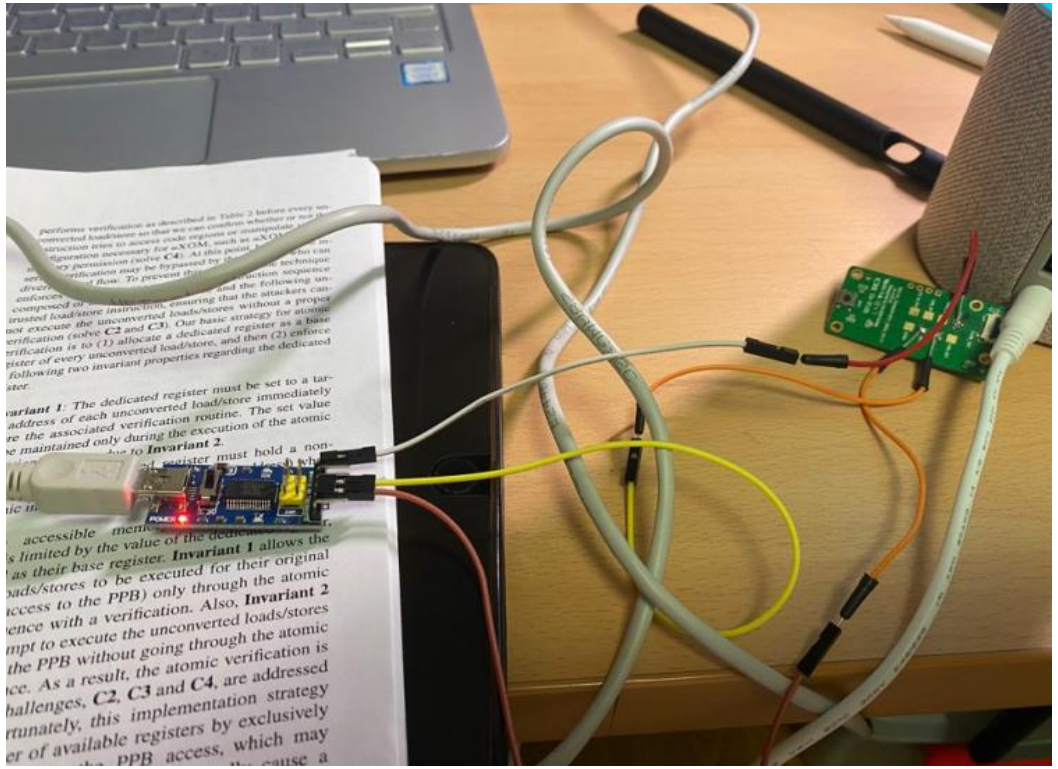
- Python 스크립트를 작성하여 분석한 결과, 부팅 메시지가 정상적으로 출력되는 것을 확인

```
PS C:\Users\wiwiw\Desktop\do> python .\test.py
CPU: Exynos7570 Rev0.2 [Samsung SOC on SMP Platform Base on ARM CortexA53]

Board: ESPRESSO7570
DRAM:  1 GiB

TrustZone Enabled BSP
BL1 version: 15112412

szWaferID: N81DX
nAsv_TABLE: 6
nAsv_CPUCL0: 7
nAsv_G3D: 7
nAsv_MIF: 7
nAsv_INT: 7
nAsv_CAM_DISP: 7
nAsv_CP: 7


Checking Boot Mode ... EMMC

EL3 Monitor information:
Larimar-W-MMB29K-6B10ES5- ef52ce4 3a5b71d 6dc1de8 6f4982d 1925431 swp aarch64-linux-gnu-gc
[SECLIB] JavaW-MMB29K-6C2-80-g14d5 14d5b3a fd9e42d 6fad68a 11afa5f 7850eeb rhyme aarch64-

MMC:   S5P_MSHC0: 0
MMC Device 0: 3.6 GiB
```

[출력된 부팅 메시지]

UART 포트 접속 및 분석

- 방법4: FT232RL USB to UART converter
  - Baud rate을 115200으로 설정
  - 연결 케이블 재확인



UART 포트 접속 및 분석

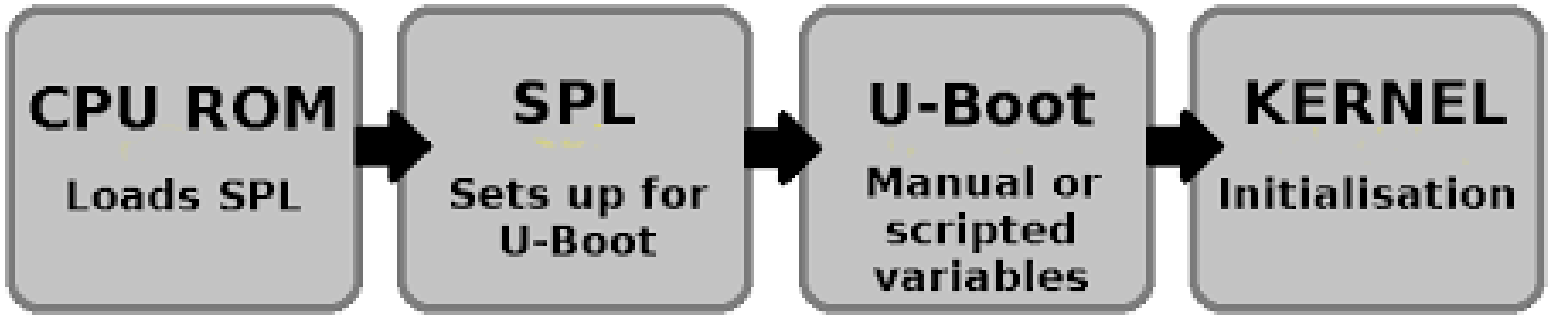
◆ 방법4: FT232RL USB to UART converter

- Uboot shell 획득 성공

```
Core stat at previous(IRAM)
Core0: Unknown: 0x1
Core1: Hotplug
Core2: Hotplug
Core3: Hotplug
Core stat at previous(KERNEL)
Core0: Unknown Power Stat: 0xeaaabaaa

Core1: Unknown Power Stat: 0xa7aabfbb

Core2: Unknown Power Stat: 0xafce7aeb

Core3: Unknown Power Stat: 0xaea7ffd9

Enable dumpGPR - f01
ramdisk flag: 0x0
RPMB partition OPEN Success.!!
Reading of the Write Counter value request
Reading of the Write Counter value request (Swapped)
Reading of the Write Counter value response
Reading of the Write Counter value response (Swapped)
RPMB: Read write counter 0
RPMB: Read write counter address 0
RPMB: Read write counter block count 0
RPMB: Read write counter result 0
RPMB: Read write counter response 200
RPMB: MAC
RPMB: key already programmed
RPMB partition CLOSE Success.!!
RPMB: key blocking: success
RPMB: hmac blocking: success
Net:   No ethernet found.
Hit enter key to stop autoboot:  0
Captured Input (ASCII: 0x0D)

ESPRESSO7570 #
ESPRESSO7570 #
ESPRESSO7570 #
ESPRESSO7570 #
ESPRESSO7570 #
ESPRESSO7570 #
ESPRESSO7570 # ls
Unknown command 'ls' - try 'help'
ESPRESSO7570 #
```

Uboot Shell

- Uboot는 디바이스가 부팅되는 과정 중 하나
- Uboot 명령어를 이용하여
  - Kernel이 실행되기 전에 실행되고, 메모리를 직접 수정하거나 읽기 가능
  - 임의 주소의 메모리를 읽어 덤프 가능 (펌웨어 덤프 가능)



[디바이스 부팅 과정]

펌웨어 업데이트 패킷 분석

◆ NUGU 스피커 펌웨어 업데이트 패킷

- 암호화되어 있음.

tcp.stream eq 21

No.	Time	Source	Destination	Protocol	Length	Info
1250	22.060778	23.59.72.16	192.168.137.…	TCP	1354	443 → 39711 [ACK] Seq=724800 Ack=519 Win=31104 Len=1288 TSval=4258214796 TSecr=4294940523 [TCP
1251	22.060849	23.59.72.16	192.168.137.…	TCP	1354	443 → 39711 [ACK] Seq=726088 Ack=519 Win=31104 Len=1288 TSval=4258214796 TSecr=4294940523 [TCP
1252	22.060905	23.59.72.16	192.168.137.…	TCP	1354	443 → 39711 [ACK] Seq=727376 Ack=519 Win=31104 Len=1288 TSval=4258214796 TSecr=4294940523 [TCP
1253	22.060937	23.59.72.16	192.168.137.…	TCP	1354	443 → 39711 [ACK] Seq=728664 Ack=519 Win=31104 Len=1288 TSval=4258214796 TSecr=4294940523 [TCP
1254	22.060971	23.59.72.16	192.168.137.…	TCP	1354	443 → 39711 [ACK] Seq=729952 Ack=519 Win=31104 Len=1288 TSval=4258214796 TSecr=4294940523 [TCP
1255	22.061009	23.59.72.16	192.168.137.…	TCP	1354	443 → 39711 [ACK] Seq=731240 Ack=519 Win=31104 Len=1288 TSval=4258214797 TSecr=4294940523 [TCP
1256	22.061040	23.59.72.16	192.168.137.…	TCP	1354	443 → 39711 [ACK] Seq=732528 Ack=519 Win=31104 Len=1288 TSval=4258214797 TSecr=4294940523 [TCP
1257	22.061105	23.59.72.16	192.168.137.…	TCP	1354	443 → 39711 [ACK] Seq=733816 Ack=519 Win=31104 Len=1288 TSval=4258214797 TSecr=4294940523 [TCP
1258	22.061525	23.59.72.16	192.168.137.…	TCP	1354	443 → 39711 [ACK] Seq=735104 Ack=519 Win=31104 Len=1288 TSval=4258214798 TSecr=4294940523 [TCP
1259	22.061602	23.59.72.16	192.168.137.…	TCP	1354	443 → 39711 [ACK] Seq=736392 Ack=519 Win=31104 Len=1288 TSval=4258214798 TSecr=4294940523 [TCP
1260	22.061635	23.59.72.16	192.168.137.…	TCP	1354	443 → 39711 [ACK] Seq=737680 Ack=519 Win=31104 Len=1288 TSval=4258214798 TSecr=4294940523 [TCP

펌웨어 분석 요약

펌웨어 획득
제조사 홈페이지에서 다운로드
패킷 스니핑
디버깅 포트를 통한 펌웨어 덤프
...
펌웨어 구조 분석
섹션 별 offset 확인
파일 시스템 확인
파일 시스템 분석
펌웨어 분석
검증 벡터 선정
사용자 인터페이스 분석
네트워크 처리 부분 분석
외부 입력 데이터 처리 부분 분석
펌웨어 무결성 처리 부분 분석
정적/동적 분석
바이너리 파일 분석과 유사
디버깅 코드 존재 여부
하드코딩된 패스워드
취약한 함수 사용 여부
...

Firmware Analysis

- ‘strings’ command
  - options
    - -n : 최소 글자 수
    - -e : 인코딩 타입
    - -tx, -to : 문자열의 오프셋 출력. x hexadecimal, o octal

```
$ strings -n16 -tx file.bin
de1d73 vl1T-W4m% ]e7 ^")
14b3b12 K,E>$r!!qxc`    a~S
15715a8 hX3Y@<-Gb$r+G9[j
19717f0 hg9Dfs[31+.|~#y*4
3a223b5 v?_-=jO ?0n>#@[D
417fec4 s]pD(6X#_tD&-NN-
47667a1 dAsMJjD#=+x'LG4<b7
4d55401 =GKw]I6VCDuTGvsv
511ad94 HelloFirmwareFans
53ef9cc %z.rkn'-z:gVUUl1-i
548b9e0 oelinux123
5d1c7cf P~7^SLD0njEo:ALa+
```

binwalk

- 설치 방법(linux)
  - sudo apt install binwalk
- 파일 탐색기에서 root 디렉토리로 가기
  - \\wsl$
- wsl
  - 확인
    - wsl –l -v
  - Ubuntu version 확인
    - lsb_release –a
  - 백업
    - wsl –export Ubuntu-20.04 backup.tar
  - 복구
    - wsl –import Ubuntu-20.04 (install location) (file location)

binwalk

- 펌웨어 이미지 확인
  - binwalk ****.bin
  - 커널 이미지와 주소 확인
- 커널 이미지 추출
  - dd if=****.bin skip=시작주소 bs=1 of=kernel.lzma
  - if : input file
  - bs : block size
  - of : output file
- 커널 파일 확인
  - file kernel.lzma
  - unlzma kernel.lzma
  - file kernel
  - binwalk kernel
    - kernel version과 kernel 데이터 확인
    - dd 명령어 반복
- sasquatch 설치
  - git clone https://github.com/devttys0/sasquatch
  - 설치 후 binwalk 실행

firmware modification kit (FMK)

- 사이트
  - https://www.kali.org/tools/firmware-mod-kit/#firmware-mod-kit
  - https://github.com/rampageX/firmware-mod-kit
- 설치
  - Ubuntu 20.04 and newer
    - sudo apt-get install git build-essential zlib1g-dev liblzma-dev python3-magic autoconf python-is-python3
    - 에러 발생할 경우, sudo apt update 실행 후 재시도
  - Git Clone
    - git clone https://github.com/rampageX/firmware-mod-kit
- 펌웨어 추출
  - ./extract-firmware.sh ****.bin

Firmware analysis homework

- Procedure
  - Download a firmware from the Internet
  - Try to fine hardcoded passwords
  - Extract the file system of the firmware
  - Analyze the file system
    - Directory hierarchy
    - Analyze some important(?) files
- Your report must include
  - Where/how you download the firmware
  - Screenshots of the above procedure steps
  - Detailed explanation of the screenshots
  - …

Q&A